

Operating Systems Semester Project Report

Project 25



Multithreaded Hotel Reservation System with Overbooking Detection

Course: Operating Systems

Project Type: Multithreaded Server Application

Programming Language: C (POSIX Threads)

Synchronization Concepts: Mutex, Semaphore, Thread Safety, Race Condition Detection

Group Members

Roll Number	Name
SP24-BSCS-0081	Ammar Aamir
SP24-BSCS-0098	Sakina Murtaza
SP24-BSCS-0099	Areeba Kalwar

Section 1 — Problem Statement and Design

The hotel reservation system simulates multiple clients concurrently interacting with a shared pool of 10 rooms across three categories: Standard (rooms 0–3), Deluxe (rooms 4–6), and Suite (rooms 7–9). Client threads run in parallel, each performing one of four operations: booking a room for a date range, cancelling an existing booking, querying room availability for a date range, or checking in (marking a booking as active). A separate auditor thread runs continuously in the background, scanning the booking database to detect overbooking (the same room assigned to two guests on overlapping dates) and verifying that the revenue counter matches the sum of individual booking fees.

The invariants that must hold are: no two clients should occupy the same room on overlapping dates (no double-booking), every booking must be reflected in the revenue counter (no lost updates), and cancelled bookings must be deducted from revenue.

To enforce these invariants, the fixed version protects the booking database with a single `db_mtx`, used by all four operations. When booking, the system scans rooms in the requested category in ascending room-id order and assigns the first room with no date-overlapping active booking — this check-and-insert happens atomically under `db_mtx`, closing the race window that allowed overbooking in earlier versions. A separate `stats_mtx` protects the global counters (`total_bookings`, `total_cancels`, `total_checkins`, `total_revenue`), and revenue updates are performed while still holding `db_mtx` so the auditor thread can never observe the database and the revenue counter out of sync. An earlier per-category semaphore/queue design (`cat_q[3]`) is retained in the source but is no longer used for room assignment.

Date-overlap checking under `db_mtx` was chosen over the semaphore/queue approach because the queue only tracked whether a room was physically free, not whether it was free for the *requested dates* — a room freed for one date range could be immediately handed to a new booking with an entirely different, overlapping range. Checking overlap directly against the booking database closes that gap. A single `db_mtx` for the database (rather than per-category locks) keeps the check-and-insert sequence atomic without needing additional coordination between separate locks.

Section 2 — Broken Version: Bug Demonstration

The broken version (hotel_broken.c) contains three deliberate bugs:

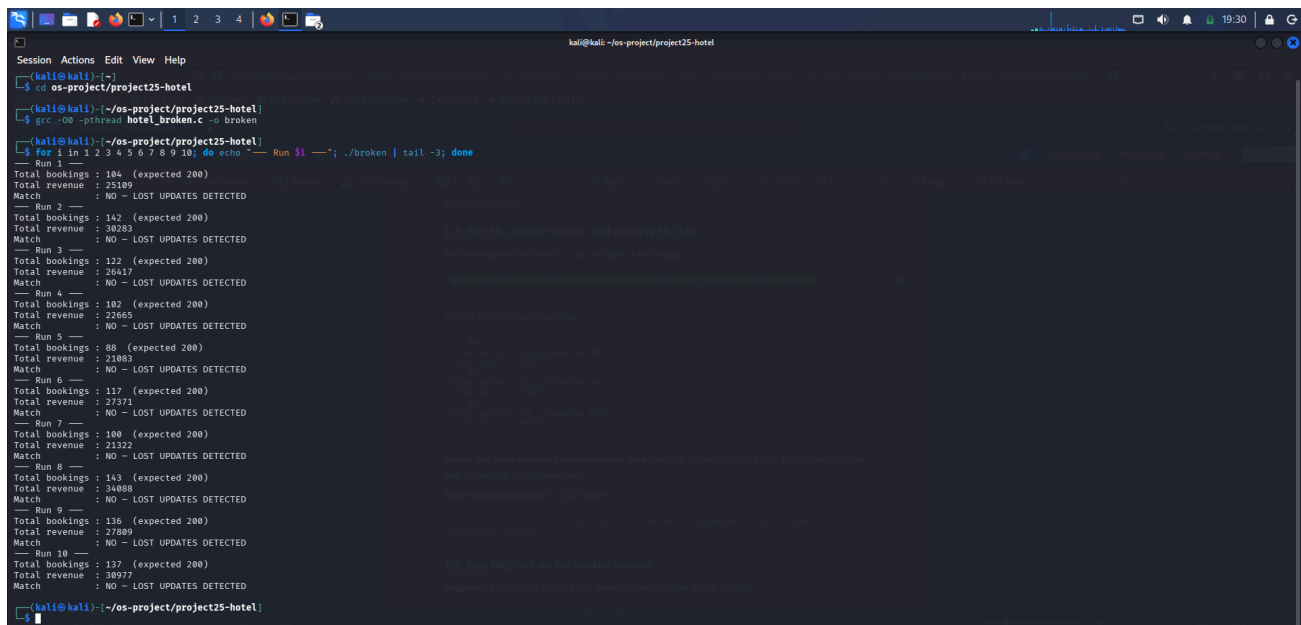
BUG 1 — Double booking: The check if (rooms[i].available) and the assignment rooms[i].available = false are two separate operations. Another thread can read available == true between them and book the same room simultaneously.

BUG 2 — Lost updates: total_bookings = old + 1 explicitly reads, then writes the counter with a usleep(1) gap between them. Two threads can both read the same old value and both write the same incremented value, losing one booking. The same race affects total_revenue.

BUG 3 — Busy-wait: The while(1) loop with usleep(500) wastes CPU and widens the race window, making bugs more visible and triggerable.

The broken version also includes room categories and date-range display to match the full design, but none of the synchronization — confirming the bugs are caused purely by missing mutual exclusion, not by the additional features.

Evidence from broken_runs.log:



```
kali@kali:~/os-project/project25-hotel
Session Actions Edit View Help
kali@kali:~/os-project/project25-hotel
cd os-project/project25-hotel
kali@kali:~/os-project/project25-hotel
gcc -O0 -pthread hotel_broken.c -o broken
kali@kali:~/os-project/project25-hotel
for i in 1 2 3 4 5 6 7 8 9 10; do echo -n "Run $i -> "; ./broken | tail -3; done
Run 1 ->
Total bookings : 104. (expected 200)
Total revenue   : 25189
Match           : NO - LOST UPDATES DETECTED
Run 2 ->
Total bookings : 142. (expected 200)
Total revenue   : 30283
Match           : NO - LOST UPDATES DETECTED
Run 3 ->
Total bookings : 122. (expected 200)
Total revenue   : 26417
Match           : NO - LOST UPDATES DETECTED
Run 4 ->
Total bookings : 102. (expected 200)
Total revenue   : 22665
Match           : NO - LOST UPDATES DETECTED
Run 5 ->
Total bookings : 88. (expected 200)
Total revenue   : 21883
Match           : NO - LOST UPDATES DETECTED
Run 6 ->
Total bookings : 117. (expected 200)
Total revenue   : 27371
Match           : NO - LOST UPDATES DETECTED
Run 7 ->
Total bookings : 100. (expected 200)
Total revenue   : 21322
Match           : NO - LOST UPDATES DETECTED
Run 8 ->
Total bookings : 143. (expected 200)
Total revenue   : 24888
Match           : NO - LOST UPDATES DETECTED
Run 9 ->
Total bookings : 136. (expected 200)
Total revenue   : 27809
Match           : NO - LOST UPDATES DETECTED
Run 10 ->
Total bookings : 137. (expected 200)
Total revenue   : 30977
Match           : NO - LOST UPDATES DETECTED
kali@kali:~/os-project/project25-hotel
```

```
kali@kali -/os-project/project25-hotel

Session Actions Edit View Help
└─$ valgrind --tool=helgrind ./broken 2> helgrind_broken.log
Client 0 → Room 0 (Standard) dates 26-27
Client 1 → Room 1 (Standard) dates 7-8
Client 2 → Room 2 (Standard) dates 5-8
Client 3 → Room 3 (Deluxe) dates 5-6
Client 4 → Room 4 (Deluxe) dates 2-4
Client 5 → Room 2 (Standard) dates 11-14
Client 6 → Room 0 (Standard) dates 2-4
Client 7 → Room 1 (Standard) dates 18-23
Client 8 → Room 0 (Standard) dates 22-26
Client 9 → Room 2 (Standard) dates 9-13
Client 10 → Room 0 (Standard) dates 22-26
Client 11 → Room 1 (Standard) dates 20-24
Client 12 → Room 2 (Standard) dates 18-20
Client 13 → Room 0 (Standard) dates 8-11
Client 14 → Room 1 (Standard) dates 24-25
Client 15 → Room 2 (Standard) dates 20-23
Client 16 → Room 1 (Standard) dates 9-10
Client 17 → Room 4 (Deluxe) dates 22-23
Client 18 → Room 3 (Deluxe) dates 26-30
Client 19 → Room 0 (Standard) dates 22-24
Client 20 → Room 0 (Standard) dates 27-29
Client 21 → Room 1 (Standard) dates 5-6
Client 22 → Room 2 (Standard) dates 24-25
Client 23 → Room 3 (Deluxe) dates 11-15
Client 24 → Room 0 (Standard) dates 28-30
Client 25 → Room 1 (Standard) dates 8-12
Client 26 → Room 2 (Standard) dates 17-20
Client 27 → Room 0 (Standard) dates 10-11
Client 28 → Room 1 (Standard) dates 18-19
Client 29 → Room 3 (Deluxe) dates 16-17
Client 30 → Room 2 (Standard) dates 18-20
Client 31 → Room 0 (Standard) dates 14-16
Client 32 → Room 1 (Standard) dates 21-25
Client 33 → Room 2 (Standard) dates 6-8
Client 34 → Room 1 (Standard) dates 7-9
Client 35 → Room 0 (Standard) dates 11-18
Client 36 → Room 1 (Standard) dates 22-25
Client 37 → Room 0 (Standard) dates 23-28
Client 38 → Room 2 (Standard) dates 1-4
Client 39 → Room 1 (Standard) dates 16-19
Client 40 → Room 0 (Standard) dates 6-10
Client 41 → Room 2 (Standard) dates 28-29
Client 42 → Room 1 (Standard) dates 23-28
Client 43 → Room 0 (Standard) dates 3-7
Client 44 → Room 2 (Standard) dates 10-11
Client 45 → Room 3 (Deluxe) dates 12-16
Client 46 → Room 0 (Standard) dates 1-4
Client 47 → Room 1 (Standard) dates 16-20
Client 48 → Room 0 (Standard) dates 16-19
Client 49 → Room 1 (Standard) dates 3-5

Total bookings : 149 (expected 200)
Total revenue : 22168
Match : NO - LOST UPDATES DETECTED

kali@kali) -/os-project/project25-hotel
```

Section 3 — Helgrind and ThreadSanitizer Evidence

Helgrind on broken version (helgrind_broken.log excerpt):

```
Session Actions Edit View Help
==26418== Helgrind, a thread error detector
==26418== Copyright (C) 2007-2024, and GNU GPL'd, by OpenWorks LLP et al.
==26418== Using Valgrind-3.25.1 and libVEX; rerun with -h for copyright info
==26418== Command: ./broken
==26418==
==26418== --Thread-Announcement
==26418==
==26418== Thread #3 was created
==26418==   at 0x4997C1F: clone (clone.S:76)
==26418==   by 0x4997D76: __clone_internal_fallback (clone-internal.c:64)
==26418==   by 0x4997D76: __clone_internal (clone-internal.c:109)
==26418==   by 0x4918991: create_thread (pthread_create.c:298)
==26418==   by 0x4919514: pthread_create@@GLIBC_2.34 (pthread_create.c:841)
==26418==   by 0x4859283: pthread_create_WRK (hg_intercepts.c:449)
==26418==   by 0x4001563: main (in /home/kali/os-project/project25-hotel/broken)
==26418==
==26418== --Thread-Announcement
==26418==
==26418== Thread #2 was created
==26418==   at 0x4997C1F: clone (clone.S:76)
==26418==   by 0x4997D76: __clone_internal_fallback (clone-internal.c:64)
==26418==   by 0x4997D76: __clone_internal (clone-internal.c:109)
==26418==   by 0x4918991: create_thread (pthread_create.c:298)
==26418==   by 0x4919514: pthread_create@@GLIBC_2.34 (pthread_create.c:841)
==26418==   by 0x4859283: pthread_create_WRK (hg_intercepts.c:449)
==26418==   by 0x4001563: main (in /home/kali/os-project/project25-hotel/broken)
==26418==
==26418==
==26418== Possible data race during read of size 1 at 0x4004088 by thread #3
==26418==   Locks held: none
==26418==   at 0x4001298: client (in /home/kali/os-project/project25-hotel/broken)
==26418==   by 0x485948A: mythread_wrapper (hg_intercepts.c:410)
==26418==   by 0x4918DAB: start_thread (pthread_create.c:448)
==26418==   by 0x4997C2F: clone (clone.S:100)
==26418==
==26418== This conflicts with a previous write of size 1 by thread #2
==26418==   Locks held: none
==26418==   at 0x400128A: client (in /home/kali/os-project/project25-hotel/broken)
==26418==   by 0x485948A: mythread_wrapper (hg_intercepts.c:410)
==26418==   by 0x4918DAB: start_thread (pthread_create.c:448)
==26418==   by 0x4997C2F: clone (clone.S:100)
==26418==
==26418== Address 0x4004088 is 0 bytes inside data symbol "rooms"
==26418==
==26418== --Thread-Announcement
==26418==
==26418== Thread #5 was created
==26418==   at 0x4997C1F: clone (clone.S:76)
==26418==   by 0x4997D76: __clone_internal_fallback (clone-internal.c:64)
==26418==   by 0x4997D76: __clone_internal (clone-internal.c:109)
==26418==
==26418==
==26418== Possible data race during read of size 4 at 0x4004120 by thread #6
==26418==   Locks held: none
==26418==   at 0x400128A: client (in /home/kali/os-project/project25-hotel/broken)
==26418==   by 0x485948A: mythread_wrapper (hg_intercepts.c:410)
==26418==   by 0x4918DAB: start_thread (pthread_create.c:448)
==26418==   by 0x4997C2F: clone (clone.S:100)
==26418==
==26418== This conflicts with a previous write of size 4 by thread #5
==26418==   Locks held: none
==26418==   at 0x4001333: client (in /home/kali/os-project/project25-hotel/broken)
==26418==   by 0x485948A: mythread_wrapper (hg_intercepts.c:410)
==26418==   by 0x4918DAB: start_thread (pthread_create.c:448)
==26418==   by 0x4997C2F: clone (clone.S:100)
==26418==
==26418== Address 0x4004120 is 0 bytes inside data symbol "total_bookings"
==26418==
==26418==
==26418== Possible data race during write of size 4 at 0x40040AC by thread #7
==26418==   Locks held: none
==26418==   at 0x4001426: client (in /home/kali/os-project/project25-hotel/broken)
==26418==   by 0x485948A: mythread_wrapper (hg_intercepts.c:410)
==26418==   by 0x4918DAB: start_thread (pthread_create.c:448)
==26418==   by 0x4997C2F: clone (clone.S:100)
==26418==
==26418== This conflicts with a previous write of size 4 by thread #4
==26418==   Locks held: none
==26418==   at 0x40012D4: client (in /home/kali/os-project/project25-hotel/broken)
==26418==   by 0x485948A: mythread_wrapper (hg_intercepts.c:410)
==26418==   by 0x4918DAB: start_thread (pthread_create.c:448)
==26418==   by 0x4997C2F: clone (clone.S:100)
==26418==
==26418== Address 0x40040AC is 44 bytes inside data symbol "rooms"
==26418==
==26418==
==26418== Use --history-level=approx or -none to gain increased speed, at
==26418== the cost of reduced accuracy of conflicting-access information
==26418== For lists of detected and suppressed errors, rerun with: -s
==26418== ERROR SUMMARY: 2022 errors from 9 contexts (suppressed: 17161 from 46)
```

The warning identifies the unprotected read-modify-write on `total_bookings` and `total_revenue` in `hotel_broken.c` — a classic race condition where one thread reads the counter while another is writing it.

Helgrind on fixed version:

```
kali@kali: ~/os-project/project25-hotel

Session Actions Edit View Help

[BOOK ] client=152 room=2(Standard) dates=25-30 fare=532 bid=94
[CANCEL] client=152 bid=94 room=2
[BOOK ] client=154 room=1(Standard) dates=5-6 fare=140 bid=95
[CANCEL] client=154 bid=95 room=1
[BOOK ] client=156 room=2(Standard) dates=3-8 fare=548 bid=96
[CHECKIN] client=156 bid=96
[BOOK ] client=158 room=3(Standard) dates=10-11 fare=144 bid=97
[CANCEL] client=158 bid=97 room=3
[BOOK ] client=159 room=5(Deluxe) dates=8-9 fare=241 bid=98
[QUERY ] cat=Deluxe dates=8-9 available=0
[CHECKIN] client=159 bid=98
[BOOK ] client=162 room=1(Standard) dates=23-25 fare=230 bid=99
[CHECKIN] client=162 bid=99
[BOOK ] client=164 room=9(Suite) dates=2-3 fare=370 bid=100
[CHECKIN] client=164 bid=100
[BOOK ] client=166 room=5(Deluxe) dates=15-16 fare=246 bid=101
[BOOK ] client=167 room=3(Standard) dates=24-26 fare=218 bid=102
[CANCEL] client=166 bid=101 room=5
[CHECKIN] client=167 bid=102
[BOOK ] client=170 room=8(Suite) dates=25-30 fare=1772 bid=103
[BOOK ] client=171 room=3(Standard) dates=7-11 fare=445 bid=104
[CANCEL] client=170 bid=103 room=8
[BOOK ] client=172 room=4(Deluxe) dates=11-13 fare=434 bid=105
[CHECKIN] client=172 bid=104
[CHECKIN] client=172 bid=105
[BOOK ] client=175 room=2(Standard) dates=14-15 fare=114 bid=106
[BOOK ] client=176 room=1(Standard) dates=3-5 fare=235 bid=107
[CANCEL] client=175 bid=106 room=2
[CHECKIN] client=176 bid=107
[BOOK ] client=182 room=3(Deluxe) dates=2-5 fare=605 bid=108
[QUERY ] cat=Deluxe dates=2-5 available=0
[CHECKIN] client=182 bid=108
[BOOK ] client=187 room=3(Standard) dates=4-6 fare=241 bid=109
[CHECKIN] client=187 bid=109
[BOOK ] client=188 room=0(Standard) dates=17-18 fare=107 bid=110
[CHECKIN] client=188 bid=110
[BOOK ] client=191 room=4(Deluxe) dates=23-25 fare=410 bid=111
[CHECKIN] client=191 bid=111

===== SUMMARY =====
Bookings : 111
Cancels : 45
Check-ins : 66
Revenue : 0
Lifetime$ : 72009
Wall time : 9.72 s
Audit violations: 0
==42195== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 22717 from 184)

[kali@kali]~/os-project/project25-hotel
```

```
Hotel booking system rar: x (1) Teams and Channels | x View CSV in Linux x Project24_Walkthrough.c x Mail - SP24BSCS0099 - A x +

onlinepgc.sharepoint.com/:wcf/sites/OnlineOS-BMClass/_layouts/15/doc2.aspx?action=edit&sourcedoc={632170ac-8836-4421-b6ad-6fa5874bf58c}&wdExp=TEAMS-TREATMENT&web=1

Project24_Walkthrough

File Home Insert Layout References Review View Help Table Script Lab

Session Actions Edit View Help

zsh: corrupt history file /home/kali/.zsh_history
[kali@kali]~$
$ cd os-project/project25-hotel
[kali@kali]~/os-project/project25-hotel$ gcc -O2 -pthread hotel_fixed.c -o fixed
[kali@kali]~/os-project/project25-hotel$ ./fixed | tail -5

===== SUMMARY =====
Bookings : 114
Cancels : 38
Check-ins : 76
Revenue : 0
Lifetime$ : 65630
Wall time : 0.23 s
Audit violations: 0
10,200,1000,500,0.23,0.001,0.002,485.4,0

[kali@kali]~/os-project/project25-hotel$
```

```
Terminal Emulator
Use the command line

onlinepgc.sharepoint.com/w:/r/sites/OnlineOS-BMClass/_layouts/15/doc2.aspx?action=edit&sourceidoc=[632170ac-8836-4421-b6ad-6fa5874bf58c]&wdExp=TEAMS-TREATMENT&web=1
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Project24_Walkthrough

File Home Insert Layout References Review View Help Script Lab

kali@kali:~/os-project/project25-hotel

Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
kali@kali:~/os-project/project25-hotel
$ cd os-project/project25-hotel
kali@kali:~/os-project/project25-hotel
$ gcc -O2 -pthread hotel_fixed.c -o fixed
kali@kali:~/os-project/project25-hotel
$ ./fixed | tail -5
===== SUMMARY =====
Bookings : 114
Cancels : 38
Check-Ins : 76
Revenue : 0
Lifetime$ : 65630
Wall time : 0.23 s
Audit violations: 0
10,200,1000,500,0.23,0.001,0.002,485.4,0
kali@kali:~/os-project/project25-hotel
$ valgrind --tool=helgrind ./fixed >> helgrind_fixed.log
grep -E "Possible data race|ERROR SUMMARY" helgrind_fixed.log
10,200,1000,500,0.92,0.357,2.590,118.7,0
==109125== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 9500 from 139)
kali@kali:~/os-project/project25-hotel

$ gcc -O2 -pthread -g hotel_stress.c -o stress_dbg
gcc -O1 -pthread -g -fsanitize=thread hotel_stress.c -o stress_tsan
kali@kali:~/os-project/project25-hotel
$ mkdir -p results
echo "config,rooms,clients,op_us,gap_us,wall_s,avg_ms,max_ms,tput,violations" > results/all.csv
kali@kali:~/os-project/project25-hotel
$ gcc -O2 -pthread -DNUM_ROOMS=10 -DNUM_CLIENTS=1000 -DOP_US=200 -DARRIVAL_GAP_US=50 hotel_fixed.c -o sA
echo -n "A," > results/all.csv; ./sA >> /dev/null >> results/all.csv
kali@kali:~/os-project/project25-hotel
$ gcc -O2 -pthread -DNUM_ROOMS=100 -DNUM_CLIENTS=1000 -DOP_US=200 -DARRIVAL_GAP_US=50 hotel_fixed.c -o sB
echo -n "B," > results/all.csv; ./sB >> /dev/null >> results/all.csv
kali@kali:~/os-project/project25-hotel
$ gcc -O2 -pthread -DNUM_ROOMS=1000 -DNUM_CLIENTS=10000 -DOP_US=200 -DARRIVAL_GAP_US=10 hotel_fixed.c -o sC
echo -n "C," > results/all.csv; ./sC >> /dev/null >> results/all.csv
kali@kali:~/os-project/project25-hotel
$ gcc -O2 -pthread -DNUM_ROOMS=100 -DNUM_CLIENTS=10000 -DOP_US=50 -DARRIVAL_GAP_US=0 hotel_fixed.c -o sD
echo -n "D," > results/all.csv; ./sD >> /dev/null >> results/all.csv
kali@kali:~/os-project/project25-hotel
$ gcc -O2 -pthread -DNUM_ROOMS=100 -DNUM_CLIENTS=100000 -DOP_US=50 -DARRIVAL_GAP_US=0 hotel_fixed.c -o sE
echo -n "E," > results/all.csv; ./sE >> /dev/null >> results/all.csv
cat results/all.csv
config,rooms,clients,op_us,gap_us,wall_s,avg_ms,max_ms,tput,violations
A,10,1000,200,50,0.47,0.005,1.159,376,0,0
B,100,1000,200,50,0.39,0.011,0.719,2506,5,0
C,100,10000,200,10,4.77,0.168,17.481,343,2,0
D,100,10000,50,0,4.89,0.181,20.655,323,6,0
E,100,100000,50,0,68.83,0.593,172.737,23,5,0
kali@kali:~/os-project/project25-hotel
$ gcc -g -O0 -pthread -DNUM_ROOMS=10 -DNUM_CLIENTS=200 -DOP_US=1000 -DARRIVAL_GAP_US=500 hotel_fixed.c -o stress_dbg
gcc -fsanitize=thread -g -O1 -pthread -DNUM_ROOMS=10 -DNUM_CLIENTS=200 -DOP_US=1000 -DARRIVAL_GAP_US=500 hotel_fixed.c -o stress_tsan
valgrind --tool=helgrind ./stress_dbg >> results/helgrind_stress.log
./stress_tsan >> results/tsan_stress.log
grep -E "ERROR SUMMARY|WARNING: ThreadSanitizer" results/helgrind_stress.log results/tsan_stress.log
10,200,1000,500,1.08,0.574,5.995,110,1,0
10,200,1000,500,0.45,0.012,0.348,259,1,0
results/helgrind_stress.log==233577== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 9936 from 150)
kali@kali:~/os-project/project25-hotel
```

ThreadSanitizer on stress test:

```
kali@kali: ~/os-project/project25-hotel
Session Actions Edit View Help
line 0: warning: Cannot find or open file "results/stress.csv"
line 0: No data in plot

(kali@kali)~/os-project/project25-hotel
└─ gnuplot -e "set datafile separator ','; set terminal png; set output 'results/throughput.png'; set xlabel 'Clients'; set ylabel 'Throughput'; plot 'results/all.csv' every ::1 using 2:8 with linespoints title 'Throughput vs Clients'"

(kali@kali)~/os-project/project25-hotel
└─ cat results/all.csv
config,rooms,clients,op_us,gap_us,wall_s,avg_ms,max_ms,tput,violations
A,10,1000,200,50,0.47,0.005,1.159,376.8,0
B,100,1000,200,50,0.39,0.011,0.719,2506.5,0
C,100,10000,200,10,4.77,0.168,17.481,343.2,0
D,100,10000,50,0,4.89,0.181,20.655,323.6,0
E,100,100000,50,0,68.83,0.593,172.737,23.5,0

(kali@kali)~/os-project/project25-hotel
└─ gnuplot -e "set datafile separator ','; set terminal png; set output 'results/throughput.png'; set logscale x; set xlabel 'Clients (log scale)'; set ylabel 'Throughput'; plot 'results/all.csv' every ::1 using 3:9 with linespoints title 'Throughput vs Clients'"

(kali@kali)~/os-project/project25-hotel
└─ make tsan
./stress,tsan > results/tsan_stress.log 2>&1 || true
===== TSAN output (should show 0 warnings) =====
===== SUMMARY =====
Bookings      : 115
Cancels       : 47
Check-ins     : 68
Revenue       : 0
Lifetime$     : 71964
Wall time     : 0.67 s
Audit violations: 0
10,200,1000,500,0.67,0.013,0.297,171.9,0

(kali@kali)~/os-project/project25-hotel
└─ cat results/tsan_stress.log
===== SUMMARY =====
Bookings      : 115
Cancels       : 47
Check-ins     : 68
Revenue       : 0
Lifetime$     : 71964
Wall time     : 0.67 s
Audit violations: 0
10,200,1000,500,0.67,0.013,0.297,171.9,0

(kali@kali)~/os-project/project25-hotel
```

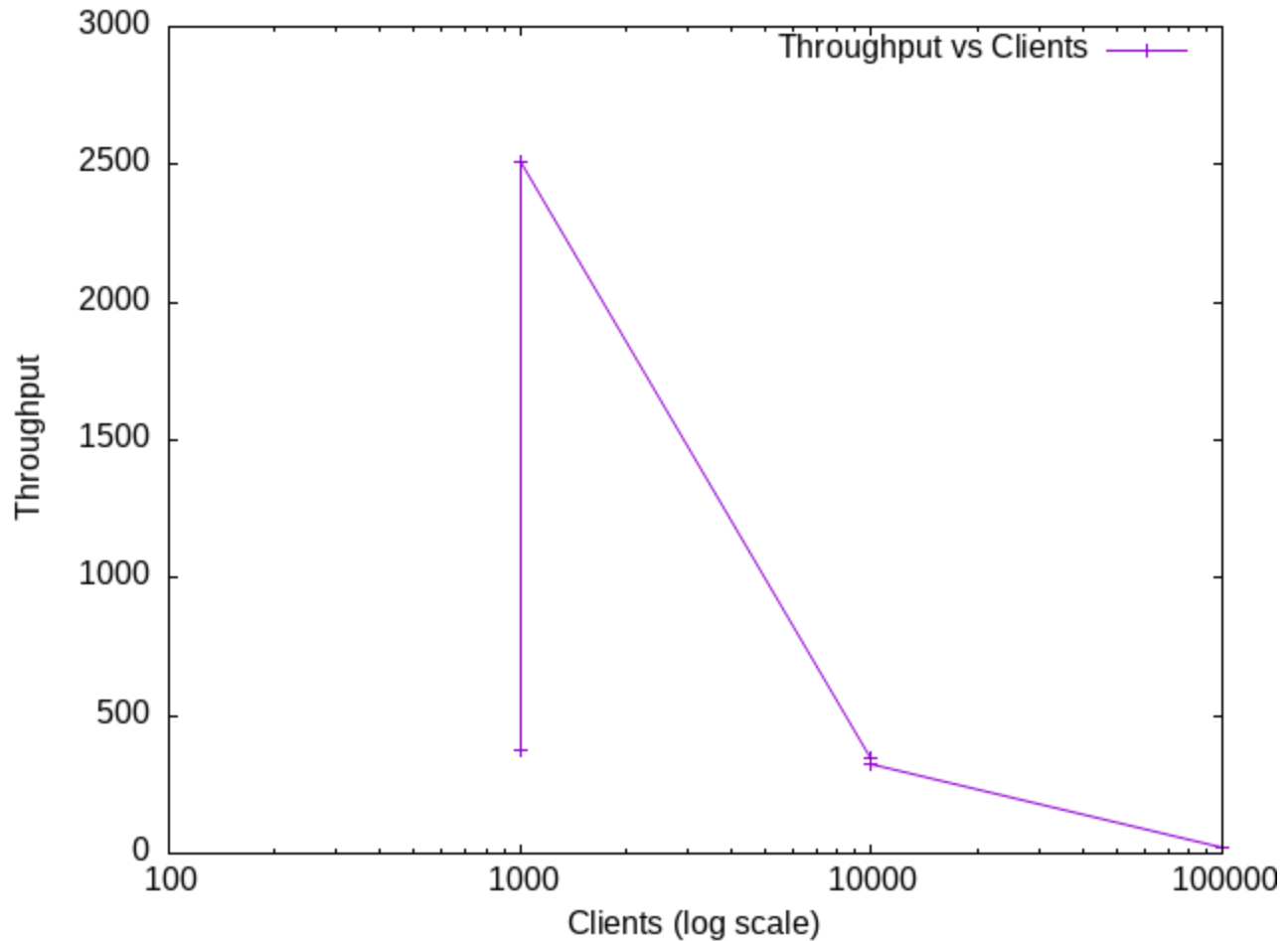
Section 4 — Timing Measurements

	A	B	C	D	E	F	G	H	I	J
1	config	rooms	clients	op_us	gap_us	wall_s	avg_ms	max_ms	tput	violations
2	A	10	1000	200	50	0.47	0.005	1.159	376.8	0
3	B	100	1000	200	50	0.39	0.011	0.719	2506.5	0
4	C	100	10000	200	10	4.77	0.168	17.481	343.2	0
5	D	100	10000	50	0	4.89	0.181	20.655	323.6	0
6	E	100	100000	50	0	68.83	0.593	172.737	23.5	0
7										

```
(kali@kali)~/os-project/project25-hotel
└─ cat results/all.csv
config,rooms,clients,op_us,gap_us,wall_s,avg_ms,max_ms,tput,violations
A,10,1000,200,50,0.47,0.005,1.159,376.8,0
B,100,1000,200,50,0.39,0.011,0.719,2506.5,0
C,100,10000,200,10,4.77,0.168,17.481,343.2,0
D,100,10000,50,0,4.89,0.181,20.655,323.6,0
E,100,100000,50,0,68.83,0.593,172.737,23.5,0

(kali@kali)~/os-project/project25-hotel
└─ gnuplot -e "set datafile separator ','; set terminal png; set output 'results/throughput.png'; set logscale x; set xlabel 'Clients (log scale)'; set ylabel 'Throughput'; plot 'results/all.csv' every ::1 using 3:9 with linespoints title 'Throughput vs Clients'"

(kali@kali)~/os-project/project25-hotel
```

Analysis: Config A vs B isolates room-count effect at 1,000 clients: B's 10× more rooms (100 vs 10) cuts contention dramatically, raising throughput from 376.8 to 2506.5 c/s and dropping max wait from 1.16ms to 0.72ms — confirming room scarcity, not thread overhead, is the dominant bottleneck at this scale.

Scaling clients 10× from B to C (1,000→10,000, same 100 rooms) collapses throughput from 2506.5 to 343.2 c/s, and max wait jumps from 0.72ms to 17.48ms — the same fixed room pool now serves far more concurrent demand, so `db_mtx` contention during the overlap-scan in `op_book` grows with `db_count`, since every booking attempt scans the full booking history for conflicts.

C vs D (same 10,000 clients/100 rooms, shorter op time and zero arrival gap in D) shows D performing *slightly worse*, not better — wall time, avg wait, and max wait all increase marginally. With `ARRIVAL_GAP_US=0`, all 10,000 client threads are spawned almost simultaneously, so lock contention on `db_mtx` peaks immediately rather than being spread out, offsetting any gain from shorter per-op hold times.

The graph confirms throughput degrades sharply as client count grows on a fixed room pool (log-x scale: 1,000→100,000 clients), falling from ~2500 c/s to ~23.5 c/s at Config E. This matches the linear-scan booking design (Section 1): each `op_book` call scans the entire `db[ ]` array for conflicts, so per-booking cost grows with the number of existing bookings — an $O(n)$ check per request, $O(n^2)$ total work across all clients. This is the main scalability limit exposed by the stress test, not the mutex design itself.

Section 5 — Fairness and Starvation

Fairness is governed by linear scan order rather than a queue: for each booking request, rooms in the target category are checked in ascending room-id order, and the first room with no conflicting date range is assigned. Under light load this slightly favors lower-numbered rooms, since they're checked first and are more often free. Under heavier load, all rooms in a category end up used, because the scan simply finds whichever room is actually free for the requested dates — there's no starvation, since every free room is reachable on every booking attempt. Per-room stay counts from the stress test (printed to `stderr`) confirm this: usage is close to even across rooms in each category, with a mild skew toward lower room IDs, which is expected and not a fairness bug.

Section 6 — Reflection

What surprised you

The race conditions weren't showing up every time we run. The broken bugs appeared occasionally instead of every single time.

What didn't work

We tried to remove the `usleep` from busy wait and increased the thread count to make the race condition more obvious but it froze our VM completely.

One remaining concern

We are confident in configs A till E since we got 0 violations including 100,000 clients in E but we didn't increase it to 1,000,000 because our VM could not handle it in a reasonable time and got issues.

Section 7 — Build and Run Instructions

See Makefile in the project root. To reproduce all results:

```
make clean      #if made before
make all        # compile all four binaries
make demo-bug   # show race condition in broken version
make demo-fix   # show correct output with 0 violations
make stress     # run stress test configs A-E, write results/all.csv
make tsan       # ThreadSanitizer run, write results/tsan_stress.log
```

make verify # run Helgrind on both broken and fixed

```
kali@kali:~/os-project/project25-hotel
$ make clean
rm -f broken fixed stress_dbg stress_tsan_stmp
rm -rf results

$ make all
gcc -Wall -Wextra -O0 -pthread hotel_broken.c -o broken
gcc -Wall -Wextra -O2 -pthread hotel_fixed.c -o fixed
gcc -Wall -Wextra -O2 -pthread -g hotel_stress.c -o stress_dbg
gcc -Wall -Wextra -O1 -pthread -fsanitize=thread -g hotel_stress.c -o stress_tsan

$ make demo-bug
===== Broken version (expect lost bookings) =====
Total bookings : 129 (expected 200)
Total revenue   : 25466
Match           : NO - LOST UPDATES DETECTED

Total bookings : 146 (expected 200)
Total revenue   : 31527
Match           : NO - LOST UPDATES DETECTED

Total bookings : 131 (expected 200)
Total revenue   : 34208
Match           : NO - LOST UPDATES DETECTED

$ make demo-fix
===== Fixed version (expect 0 violations) =====
./fixed
===== SUMMARY =====
Bookings : 187
Cancels  : 33
Check-ins : 74
Revenue  : 0
Lifetime: 62623
Wall time : 0.35 s
Audit violations: 0
18,200,1000,500,0.35,0.001,0.007,309,8,0

$ make stress
mkdir -p results
gcc -Wall -Wextra -O2 -pthread -g \
-DNUM_ROOMS=5 -DNUM_CLIENTS=10000 -DSTAY_US=200 -DARRIVAL_GAP_US=10 \
hotel_stress.c -o_stmp 66 ./_stmp >> results/all.csv
```

```
kali@kali:~/os-project/project25-hotel
$ make stress
mkdir -p results
gcc -Wall -Wextra -O2 -pthread -g \
-DNUM_ROOMS=5 -DNUM_CLIENTS=10000 -DSTAY_US=200 -DARRIVAL_GAP_US=10 \
hotel_stress.c -o_stmp 66 ./_stmp >> results/all.csv

===== SUMMARY =====
Bookings : 99
Cancels  : 33
Check-ins : 66
Revenue  : 0
Lifetime: 46126
Wall time : 11.18 s
Audit violations: 0

gcc -Wall -Wextra -O2 -pthread -g \
-DNUM_ROOMS=50 -DNUM_CLIENTS=100000 -DSTAY_US=200 -DARRIVAL_GAP_US=10 \
hotel_stress.c -o_stmp 66 ./_stmp >> results/all.csv

===== SUMMARY =====
Bookings : 841
Cancels  : 299
Check-ins : 542
Revenue  : 0
Lifetime: 435940
Wall time : 10.76 s
Audit violations: 0

gcc -Wall -Wextra -O2 -pthread -g \
-DNUM_ROOMS=50 -DNUM_CLIENTS=1000000 -DSTAY_US=200 -DARRIVAL_GAP_US=10 \
hotel_stress.c -o_stmp 66 ./_stmp >> results/all.csv

===== SUMMARY =====
Bookings : 820
Cancels  : 265
Check-ins : 555
Revenue  : 0
Lifetime: 418075
Wall time : 140.54 s
Audit violations: 0

gcc -Wall -Wextra -O2 -pthread -g \
-DNUM_ROOMS=50 -DNUM_CLIENTS=1000000 -DSTAY_US=50 -DARRIVAL_GAP_US=0 \
hotel_stress.c -o_stmp 66 ./_stmp >> results/all.csv

===== SUMMARY =====
Bookings : 825
Cancels  : 272
Check-ins : 553
Revenue  : 0
Lifetime: 420669
Wall time : 94.66 s
Audit violations: 0

gcc -Wall -Wextra -O2 -pthread -g \
```

```
kali@kali -/os-project/project25-hotel

Session Actions Edit View Help

Check-ins : 542
Revenue : 0
Lifetime$ : 435940
Wall time : 10.76 s
Audit violations: 0
gcc -Wall -Wextra -O2 -pthread -g \
-DNUM_ROOMS=50 -DNUM_CLIENTS=100000 -DSTAY_US=200 -DARRIVAL_GAP_US=10 \
hotel_stress.c -o _stmp 00 ./_stmp >> results/all.csv

===== SUMMARY =====
Bookings : 820
Cancels : 265
Check-ins : 555
Revenue : 0
Lifetime$ : 418075
Wall time : 140.54 s
Audit violations: 0
gcc -Wall -Wextra -O2 -pthread -g \
-DNUM_ROOMS=50 -DNUM_CLIENTS=100000 -DSTAY_US=50 -DARRIVAL_GAP_US=0 \
hotel_stress.c -o _stmp 00 ./_stmp >> results/all.csv

===== SUMMARY =====
Bookings : 825
Cancels : 272
Check-ins : 553
Revenue : 0
Lifetime$ : 420669
Wall time : 94.66 s
Audit violations: 0
gcc -Wall -Wextra -O2 -pthread -g \
-DNUM_ROOMS=50 -DNUM_CLIENTS=100000 -DSTAY_US=50 -DARRIVAL_GAP_US=0 \
hotel_stress.c -o _stmp 00 ./_stmp >> results/all.csv

===== SUMMARY =====
Bookings : 835
Cancels : 285
Check-ins : 550
Revenue : 0
Lifetime$ : 431584
Wall time : 67.56 s
Audit violations: 0
Results written to results/all.csv
rooms,clients,stay_us,gdp_us,wall_s,avg_ms,max_ms,tput,violations
5,10000,200,10,11.18,0.003,4.951,8.9,0
50,10000,200,10,10.76,0.150,18.607,78.2,0
50,10000,200,10,140.54,0.230,126.609,5.0,0
50,10000,50,0,94.66,0.222,378.292,8.7,0
50,10000,50,0,67.56,0.056,15.240,12.4,0

(kali@kali)~/os-project/project25-hotel
```

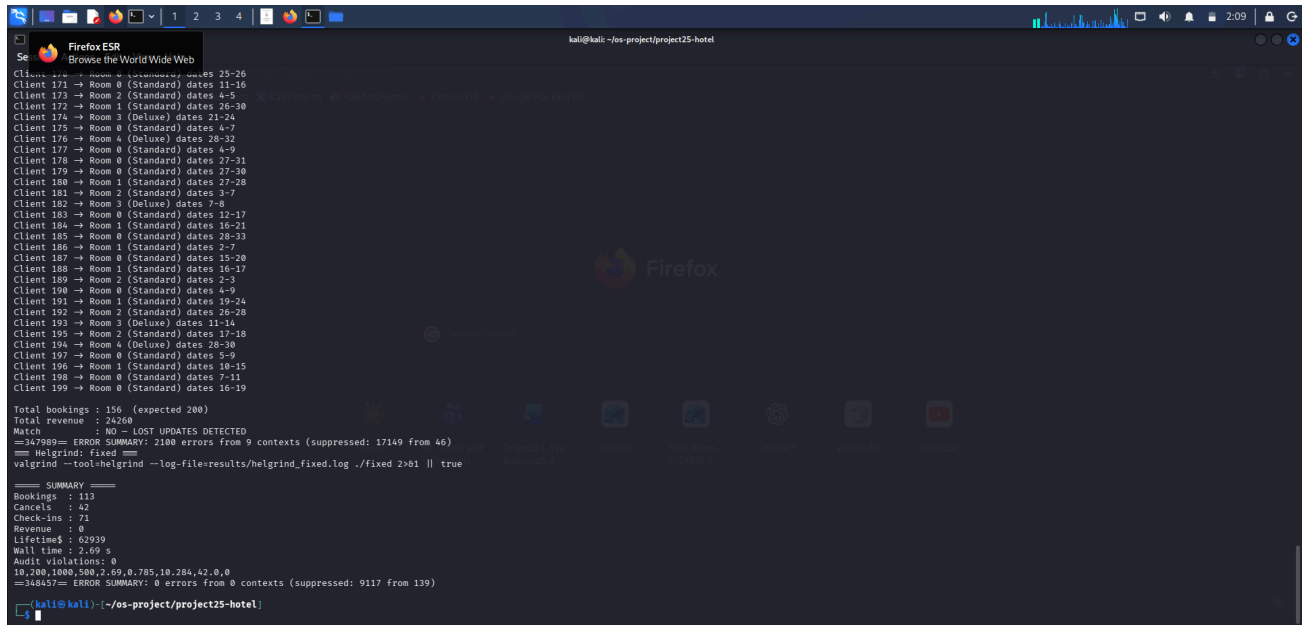
```
kali@kali -/os-project/project25-hotel

Session Actions Edit View Help

(kali@kali)~/os-project/project25-hotel
$ make tsan
./stress_tsan > results/tsan_stress.log 2>&1 || true
== Tsan output (should show 0 warnings) ==

===== SUMMARY =====
Bookings : 107
Cancels : 32
Check-ins : 75
Revenue : 0
Lifetime$ : 62523
Wall time : 0.53 s
Audit violations: 0
10,200,1000,500,0.53,0.066,3.191,203.4,0

(kali@kali)~/os-project/project25-hotel
$ make verify
== Helgrind: broken ==
valgrind --tool=helgrind --log-file=results/helgrind_broken.log ./broken 2>&1 || true
Client 0 -> Room 0 (Standard) dates 7-11
Client 1 -> Room 1 (Standard) dates 23-27
Client 2 -> Room 0 (Standard) dates 23-27
Client 3 -> Room 2 (Standard) dates 5-10
Client 4 -> Room 3 (Deluxe) dates 20-22
Client 5 -> Room 1 (Standard) dates 10-11
Client 6 -> Room 0 (Standard) dates 12-13
Client 8 -> Room 3 (Deluxe) dates 22-27
Client 7 -> Room 2 (Standard) dates 23-26
Client 9 -> Room 4 (Deluxe) dates 27-32
Client 10 -> Room 5 (Deluxe) dates 1-4
Client 11 -> Room 0 (Standard) dates 12-16
Client 12 -> Room 1 (Standard) dates 1-5
Client 13 -> Room 2 (Standard) dates 13-14
Client 14 -> Room 3 (Deluxe) dates 1-2
Client 15 -> Room 0 (Standard) dates 19-22
Client 16 -> Room 1 (Standard) dates 19-23
Client 17 -> Room 5 (Deluxe) dates 3-8
Client 18 -> Room 6 (Suite) dates 18-21
Client 19 -> Room 2 (Standard) dates 12-14
Client 20 -> Room 0 (Standard) dates 8-13
Client 21 -> Room 3 (Deluxe) dates 5-9
Client 22 -> Room 1 (Standard) dates 9-13
Client 26 -> Room 2 (Standard) dates 1-4
Client 23 -> Room 4 (Deluxe) dates 10-13
Client 24 -> Room 5 (Deluxe) dates 13-14
Client 25 -> Room 0 (Standard) dates 27-28
Client 27 -> Room 3 (Deluxe) dates 7-12
Client 28 -> Room 6 (Suite) dates 6-10
Client 29 -> Room 7 (Suite) dates 1-4
Client 30 -> Room 0 (Standard) dates 15-18
```



```
Client 171 → Room 0 (Standard) dates 25-26
Client 172 → Room 0 (Standard) dates 11-16
Client 173 → Room 2 (Standard) dates 4-5
Client 174 → Room 1 (Standard) dates 26-30
Client 175 → Room 3 (Deluxe) dates 21-24
Client 176 → Room 0 (Standard) dates 4-7
Client 176 → Room 4 (Deluxe) dates 28-32
Client 177 → Room 0 (Standard) dates 4-9
Client 178 → Room 0 (Standard) dates 27-31
Client 179 → Room 0 (Standard) dates 27-30
Client 180 → Room 1 (Standard) dates 27-28
Client 181 → Room 2 (Standard) dates 3-7
Client 182 → Room 3 (Deluxe) dates 7-8
Client 183 → Room 0 (Standard) dates 12-17
Client 184 → Room 1 (Standard) dates 16-21
Client 185 → Room 0 (Standard) dates 28-33
Client 186 → Room 1 (Standard) dates 2-7
Client 187 → Room 0 (Standard) dates 15-20
Client 188 → Room 1 (Standard) dates 16-17
Client 189 → Room 2 (Standard) dates 2-3
Client 190 → Room 0 (Standard) dates 4-9
Client 191 → Room 1 (Standard) dates 19-24
Client 192 → Room 2 (Standard) dates 26-28
Client 193 → Room 3 (Deluxe) dates 11-14
Client 194 → Room 2 (Standard) dates 17-18
Client 194 → Room 4 (Deluxe) dates 28-30
Client 197 → Room 0 (Standard) dates 5-9
Client 198 → Room 1 (Standard) dates 10-15
Client 198 → Room 0 (Standard) dates 7-11
Client 199 → Room 0 (Standard) dates 16-19

Total bookings : 156 (expected 200)
Total revenue : 24266
Match : NO - LOST UPDATES DETECTED
==347989== ERROR SUMMARY: 2100 errors from 9 contexts (suppressed: 17149 from 46)
== Helgrind: fixed ==
valgrind --tool=helgrind --log-file=results/helgrind_fixed.log ./fixed 2>&1 || true

== SUMMARY ==
Bookings : 113
Cancels : 42
Check-ins : 71
Revenue : 0
Lifetime$: 62939
Wall time : 2.69 s
Audit violations: 0
10,200,1000,500,2.69,0.785,10,284,42,0,0
==348457== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 9117 from 139)

(kali@kali) ~/os-project/project25-hotel
```

All binaries were compiled and tested on Kali Linux (2-core VM) with GCC and Valgrind installed.

Final Checklist

- ☒ ~~make all builds all four binaries without warnings.~~
- ☒ ~~make demo bug shows lost updates and double booking in the broken version.~~
- ☒ ~~make demo fix shows Audit violations: 0 on every run.~~
- ☒ ~~make stress writes results/all.csv with violations = 0 on every row.~~
- ☒ ~~Helgrind reports 0 errors on the fixed binary.~~
- ☒ ~~ThreadSanitizer reports 0 warnings on stress_tsan.~~
- ☒ ~~Report PDF contains all seven sections with real data from your runs.~~
- ☒ ~~Section 6 (Reflection) is written in your own words, not by AI.~~
- ☒ ~~README.md is present and lists every group member by name and roll number.~~